

UNIVERSIDAD AUTÓNOMA DE SANTO DOMINGO

CURSO-UASD (SANTO DOMINGO)

PRESENTACIÓN

Docente(s)

Radhames Silverio y Marlenis Concepción

Materia

Laboratorio de Lenguaje de Programación III (INF 5170)

Tema

Practica 8: Modelo MVC

Sección

01

Nombre	Matrícula
Miguel Enmanuel Castillo Garcia	DA1734

Fecha

13/10/2026

Introducción

El patrón Modelo-Vista-Controlador (MVC) es una forma de organizar el software separando las responsabilidades relacionadas con los datos y las reglas de la aplicación, la presentación de la información y la gestión de las solicitudes del usuario. Aunque surgió hace varias décadas, continúa presente en herramientas utilizadas para desarrollar aplicaciones web.

El propósito de esta investigación es explicar el origen y la finalidad de MVC, describir sus tres componentes, identificar sus ventajas y limitaciones, y comparar su uso tradicional con el de varios frameworks actuales. Finalmente, se presenta un ejemplo ilustrativo de un sistema de registro de estudiantes para reconocer cómo se distribuyen las responsabilidades sin necesidad de desarrollar una aplicación.

1. ¿Qué es el patrón MVC?

MVC, sigla de Model-View-Controller o Modelo-Vista-Controlador, es un patrón arquitectónico que divide una aplicación en tres componentes con responsabilidades diferentes. El Modelo representa los datos y las reglas del dominio; la Vista presenta la información al usuario; y el Controlador recibe las solicitudes, coordina las operaciones necesarias y determina la respuesta que se devolverá.

La separación de responsabilidades evita que toda la lógica se concentre en una sola parte del programa. De esta manera, los cambios en la interfaz pueden realizarse con menor impacto sobre las reglas de negocio, y los componentes resultan más fáciles de mantener y probar (Microsoft, s. f.).

2. Origen del modelo MVC y propósito inicial

El origen de MVC se relaciona con el trabajo de Trygve Reenskaug en Xerox PARC durante la década de 1970, en el contexto del desarrollo de interfaces gráficas y de la programación con Smalltalk. En una nota técnica de diciembre de 1979, Reenskaug documentó el uso de los conceptos Modelo, Vista y Controlador para estructurar una interfaz de usuario (Reenskaug, 1979).

Su propósito inicial era organizar la interacción entre los datos, la representación visual y las acciones del usuario. Al separar estos aspectos, era posible modificar la presentación sin tener que rehacer la representación de los datos y la lógica central. Con el tiempo, esta idea se adaptó al desarrollo de aplicaciones de escritorio y, posteriormente, a las aplicaciones web.

3. Explicación de los componentes de MVC

3.1. Modelo (Model)

Representa los datos, el estado y las reglas de negocio de la aplicación. Puede validar información, aplicar operaciones del dominio y coordinar la persistencia de los datos. En un sistema de estudiantes, por ejemplo, el modelo podría representar la matrícula, el nombre, la carrera y el correo electrónico, además de reglas como exigir una matrícula válida.

3.2. Vista (View)

Es la parte encargada de presentar la información al usuario. En una aplicación web puede ser una página con un formulario, una tabla de resultados o un mensaje de confirmación. Su responsabilidad principal es mostrar los datos de manera comprensible, evitando concentrar en ella las reglas de negocio.

3.3. Controlador (Controller)

Recibe las solicitudes o acciones del usuario, interpreta los datos recibidos, coordina el trabajo que corresponda con el modelo y selecciona la respuesta. Por ejemplo, al recibir un formulario de registro, el controlador puede solicitar la validación de los datos y decidir si presenta una confirmación o devuelve el formulario con errores.

En términos generales, el usuario interactúa con la Vista; la solicitud llega al Controlador; este coordina las operaciones con el Modelo y finalmente se genera una respuesta apropiada. La implementación concreta de ese flujo puede variar entre frameworks.

4. Ventajas y desventajas de MVC

Ventajas

- Separación de responsabilidades: cada componente tiene un propósito más claro.
- Mantenimiento: los cambios en la presentación pueden afectar menos a la lógica de negocio.
- Reutilización: el modelo puede utilizarse con distintas vistas o formas de presentar los datos.
- Pruebas: es posible probar partes de la aplicación de manera más independiente.
- Trabajo en equipo: diferentes integrantes pueden trabajar en componentes distintos con acuerdos bien definidos.

Desventajas

- Complejidad inicial: una aplicación pequeña puede requerir más archivos y estructura de la estrictamente necesaria.
- Curva de aprendizaje: es necesario comprender las responsabilidades y la comunicación entre componentes.
- Controladores sobrecargados: si se coloca demasiada lógica en ellos, se pierde parte de la separación buscada.
- Variaciones entre frameworks: el mismo concepto puede aplicarse con convenciones y estructuras diferentes.

Por tanto, MVC no garantiza por sí solo un diseño de calidad. Sus beneficios dependen de que las responsabilidades estén bien distribuidas y de que se evite acumular lógica innecesaria en la Vista o el Controlador.

5. ¿Por qué MVC sigue siendo importante en la programación actual?

MVC continúa siendo importante porque ofrece un modelo mental sencillo para comprender cómo se organiza una aplicación. La separación entre presentación, coordinación de solicitudes y lógica de negocio ayuda a que el código sea más comprensible, mantenible y comprobable.

Además, muchos frameworks web ofrecen estructuras basadas directamente en MVC o adoptan principios similares. Comprender el patrón permite al desarrollador interpretar mejor la organización de un proyecto, seguir las convenciones de una herramienta y decidir dónde debe ubicarse cada responsabilidad. No significa que todas las aplicaciones modernas deban utilizar MVC: la arquitectura adecuada depende de sus necesidades.

6. ¿Cómo se usa MVC en frameworks modernos?

En los frameworks actuales, MVC suele estar acompañado de herramientas para resolver tareas comunes, como el enrutamiento de solicitudes, la validación de datos, la vinculación de parámetros, la generación de vistas, la autenticación y las pruebas.

En ASP.NET Core MVC, por ejemplo, las solicitudes se asignan a acciones de controladores; los modelos representan datos y reglas; y las vistas pueden utilizar Razor para generar contenido HTML. La documentación de Microsoft también describe el uso de MVC para construir aplicaciones web y API (Microsoft, s. f.).

Spring MVC utiliza componentes como DispatcherServlet y controladores para procesar solicitudes web. Laravel organiza las aplicaciones mediante rutas, controladores, modelos y vistas. Ruby on Rails también estructura gran parte del código mediante el patrón MVC (Spring, s. f.; Laravel, s. f.; Ruby on Rails, s. f.).

En el caso de Django, conviene hacer una precisión: su documentación suele describir su arquitectura como Model-Template-View (MTV). Aunque sus responsabilidades guardan semejanzas con MVC, los nombres y la distribución de tareas no son idénticos; por eso no debe afirmarse que todos los frameworks implementan exactamente el mismo esquema.

7. Comparación entre MVC tradicional y frameworks actuales

Tecnología	Cómo se organiza	Particularidad
MVC tradicional	Define Modelo, Vista y Controlador como responsabilidades separadas.	Es el concepto base; no depende de un lenguaje específico.
Spring MVC	Controladores gestionan solicitudes y se integran con servicios, modelos y tecnologías de vista.	Forma parte del ecosistema Spring para aplicaciones Java.
ASP.NET Core MVC	Controladores, modelos y vistas Razor se apoyan en enrutamiento y validación.	También permite desarrollar API web.
Laravel	Rutas conducen a controladores; los modelos suelen trabajar con datos y las vistas se generan con Blade.	Framework PHP con convenciones y herramientas integradas.
Django	Organiza el código con el enfoque Model-Template-View (MTV).	Sus nombres y responsabilidades difieren parcialmente de MVC.
Ruby on Rails	Organiza la aplicación en modelos, vistas y controladores, con convenciones para acelerar el desarrollo.	Promueve convenciones y desarrollo rápido.

La comparación muestra que los frameworks no se limitan a repetir el patrón original: lo adaptan a su lenguaje, sus convenciones y sus herramientas. Por eso, aprender MVC ayuda a comprender las similitudes, pero también es necesario estudiar la documentación particular de cada framework.

8. Ejemplo ilustrativo: sistema de registro de estudiantes

Supongamos que una universidad necesita registrar estudiantes y consultar sus datos. El ejemplo siguiente identifica los tres componentes solicitados en el

mandato:

Modelo

Representa al estudiante con los campos matrícula, nombre, carrera y correo electrónico. También contiene o utiliza las reglas necesarias para comprobar que la información cumpla los requisitos y puede participar en el almacenamiento de los registros.

Vista

Presenta un formulario con los campos del estudiante y una pantalla o tabla para consultar los registros. Si ocurre un error, muestra un mensaje que ayude al usuario a corregir la información.

Controlador

Recibe la solicitud de guardar o consultar un estudiante, coordina la validación y las operaciones correspondientes, y decide qué respuesta mostrar. Si los datos son válidos, puede solicitar el registro y presentar una confirmación; si no lo son, puede devolver la vista con los errores.

Flujo resumido

1. El usuario completa el formulario en la Vista y solicita guardar el registro.
2. El Controlador recibe la solicitud y coordina el procesamiento.
3. El Modelo aplica las reglas y participa en la gestión de los datos.
4. El Controlador selecciona la respuesta y la Vista presenta el resultado.

Este ejemplo es conceptual: su objetivo es identificar las responsabilidades de MVC, no construir un sistema funcional. En una aplicación real, el almacenamiento, la validación y la coordinación pueden distribuirse también entre servicios, repositorios y otros componentes, según la arquitectura utilizada.

Conclusión personal

Considero que comprender MVC es importante porque ayuda a entender cómo se organiza una aplicación y por qué conviene separar la presentación, los datos y la

gestión de las solicitudes. Aunque es un patrón con varias décadas de existencia, sus principios siguen presentes en frameworks utilizados actualmente.

También entiendo que conocer el concepto no sustituye el aprendizaje práctico de cada tecnología. Frameworks como Spring MVC, ASP.NET Core MVC, Laravel y Ruby on Rails tienen sus propias convenciones, mientras que Django utiliza una organización relacionada, pero no idéntica. Por eso, estudiar primero los fundamentos permite abordar esas herramientas con una mejor comprensión y tomar decisiones más acertadas al desarrollar software.

Referencias bibliográficas

- Laravel. (s. f.). Documentation. https://laravel.com/docs
- Microsoft. (s. f.). Información general de ASP.NET Core MVC. Microsoft Learn. https://learn.microsoft.com/es-es/aspnet/core/mvc/overview?view=aspnetcore-10.0
- Reenskaug, T. (1979, 10 de diciembre). Models–Views–Controllers. European Smalltalk Users Group. https://doi.org/10.5281/zenodo.3676092
- Ruby on Rails. (s. f.). Getting started with Rails: Model-View-Controller basics. https://guides.rubyonrails.org/getting_started.html
- Spring. (s. f.). Spring Web MVC. Spring Framework Reference Documentation. https://docs.spring.io/spring/reference/7.1/web/webmvc.html